PROJETO AVALIATIVO · MÓDULO 2 · SEMANA 13 · SCTEC

CreditAI MLOps

Crédito decidido em milissegundos, com regras claras.

Microsserviço preditivo de concessão de crédito: um Random Forest servido por uma API REST em FastAPI, com validação Pydantic, testes automatizados e um painel em Streamlit.

90,2%	0,971	6/6	13/13
acurácia no teste	ROC-AUC	testes pytest	cenários HTTP

Jorge Luiz Susin Rodrigues
Análise Preditiva de Dados & IA · SCTEC SC · Outubro de 2026
jorgerodrigues2026.github.io/credito-dashboard

PARECER DE CRÉDITO — POST /predict

Solicitante	**Carlos Eduardo Santos**
Idade	**35 anos**
Renda mensal	**R$ 7.500**
Valor pedido	**R$ 18.000**
Finalidade	**Negócios**
Inadimplência	**Não**

720
300 score de crédito 850

Probabilidade de aprovação
95,0%

APROVADO
BAIXO RISCO

Resposta real da API para o cliente de exemplo do README.

O PROBLEMA DE NEGÓCIO

Fila manual não escala. Modelo preso em notebook também não.

Bancos, cooperativas e fintechs recebem milhares de pedidos de empréstimo ao mesmo tempo. Decidir caso a caso gera espera e critérios diferentes; um modelo que só roda no notebook não chega ao sistema do banco.

ANTES

Análise manual

- Cada pedido espera um analista disponível.
- Perfis iguais recebem decisões diferentes.
- Erros de cadastro aparecem depois da decisão.
- O modelo existe, mas não chega à produção.

DEPOIS

CreditAI MLOps

- **Mesmo critério** para todos: um pipeline treinado e versionado.
- **Resposta imediata** em POST `/predict`, com probabilidade e faixa de risco.
- **Dados inválidos barrados** (422) e cadastro incoerente sinalizado (400).
- **Analista no comando** pelo painel Streamlit.

FASE 1

Pipeline sem vazamento de dados

FASE 3

Modelo na RAM via lifespan

FASE 4

Contratos Pydantic: 422

FASE 5

Rotas REST, 400 e testes

FASE 1 · PREPARAÇÃO DO MODELO

Um pipeline único, do dado cru ao arquivo .joblib

1 **Base**
3.000 clientes
8 variáveis

→ 2 **Divisão**
70 / 15 / 15
estratificada

→ 3 **Numéricas**
mediana +
StandardScaler

→ 4 **Categóricas**
moda +
OneHotEncoder

→ 5 **Random Forest**
200 árvores
prof. 10, balanced

→ 6 **Artefato**
modelo_emprestimo
.joblib + métricas

Divisão antes de qualquer cálculo

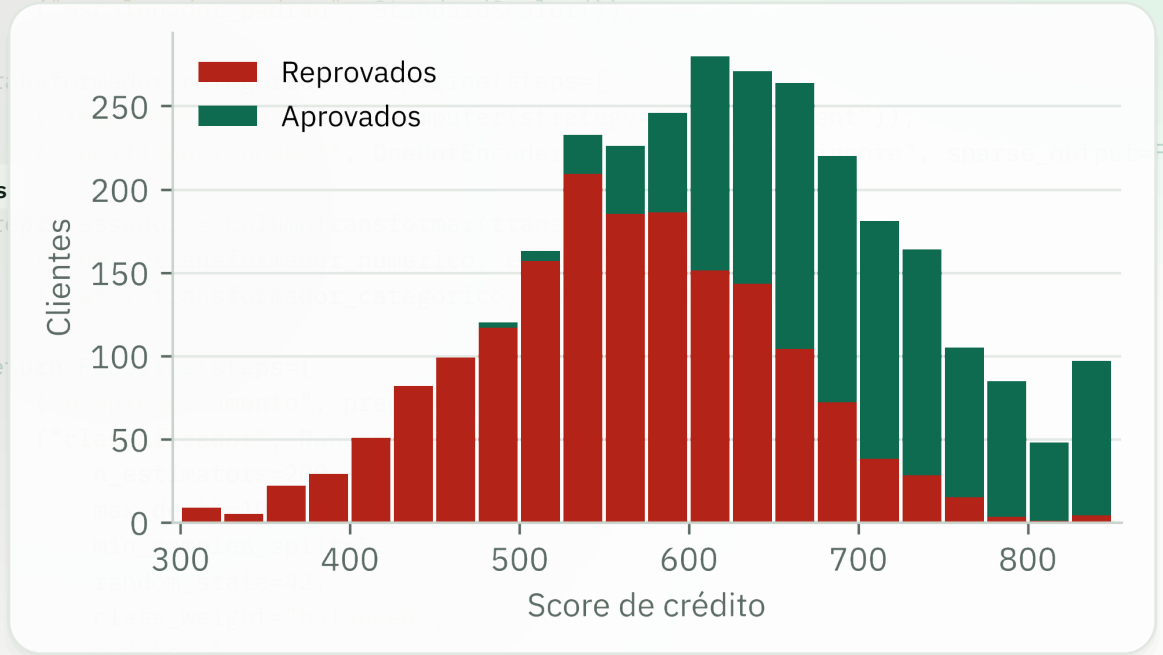


Lacunas tratadas, sem descartar clientes

`renda_mensal: 77 vazios` `tempo_emprego_anos: 110 vazios` `tipo_moradia: 98 vazios`

- ✓ **Mediana** nas numéricas: resiste a rendas muito altas.
- ✓ `handle_unknown="ignore"`: categoria nova não derruba a API.
- ✓ `class_weight="balanced"`: compensa os 57% de reprovados.
- ✓ **Artefato com metadados**: versão, algoritmo e métricas viajam juntos.

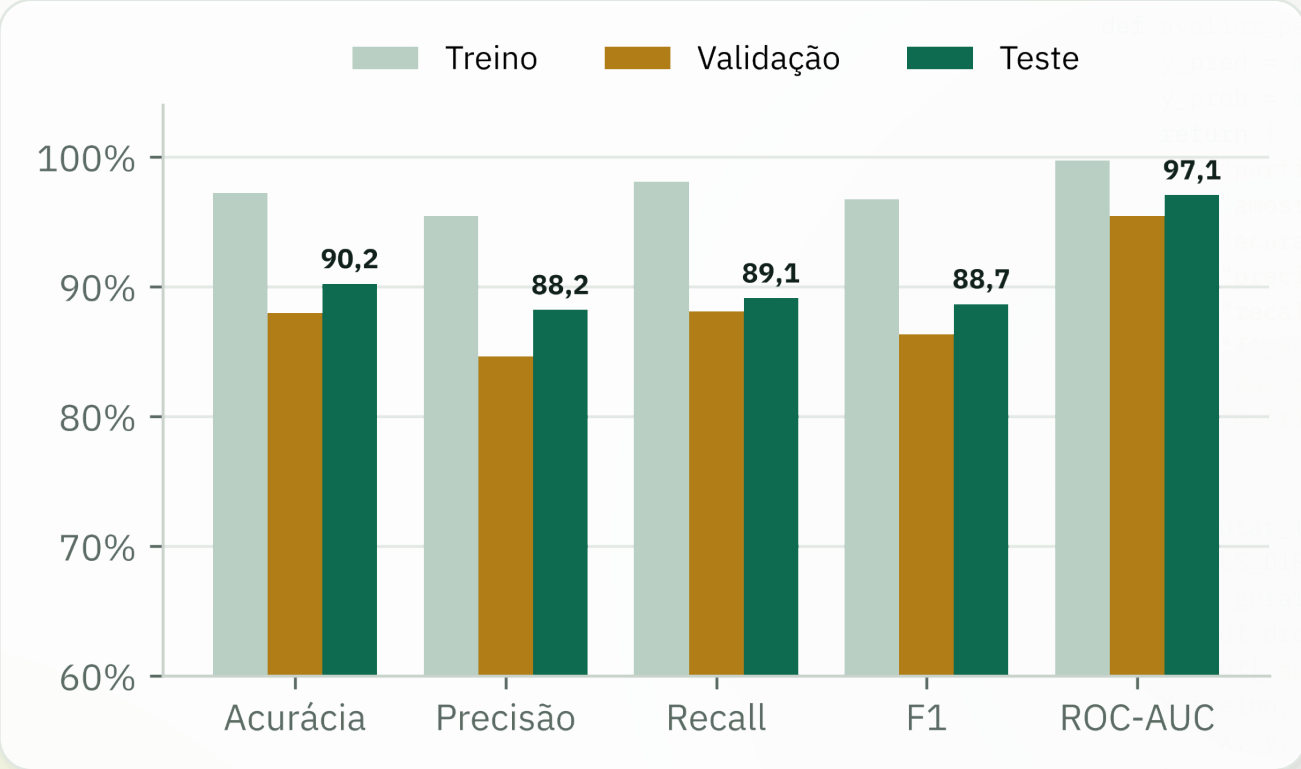
Score de crédito por decisão

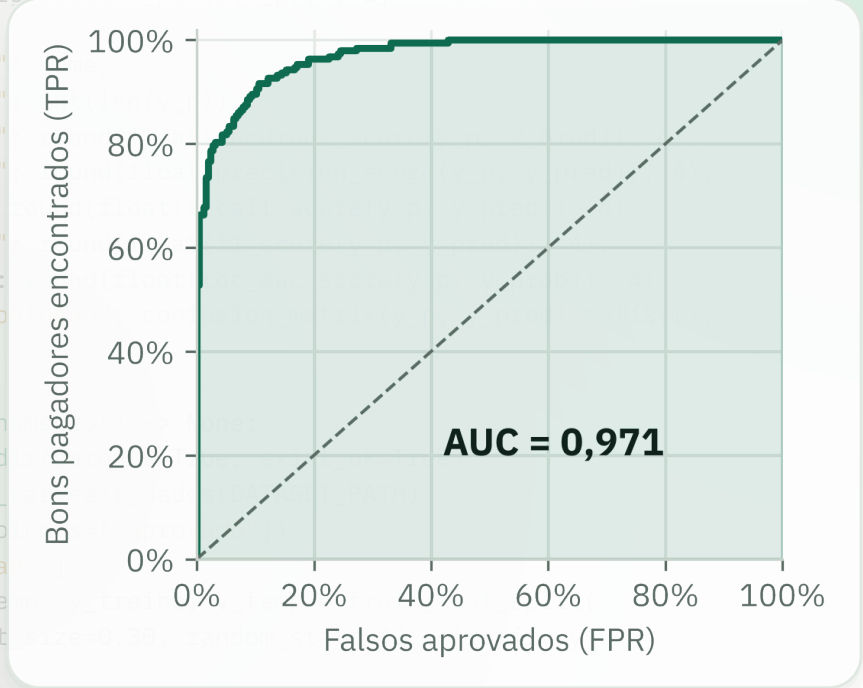


Acima de 700 pontos, quase todos são aprovados. Abaixo de 500, quase ninguém.

FASE 1 · AVALIAÇÃO

9 em cada 10 decisões certas em clientes inéditos





TESTE (450 CLIENTES)

90,2% acurácia · 0,971 ROC-AUC

TREINO X TESTE

7,0 pontos de diferença: sobreajuste leve

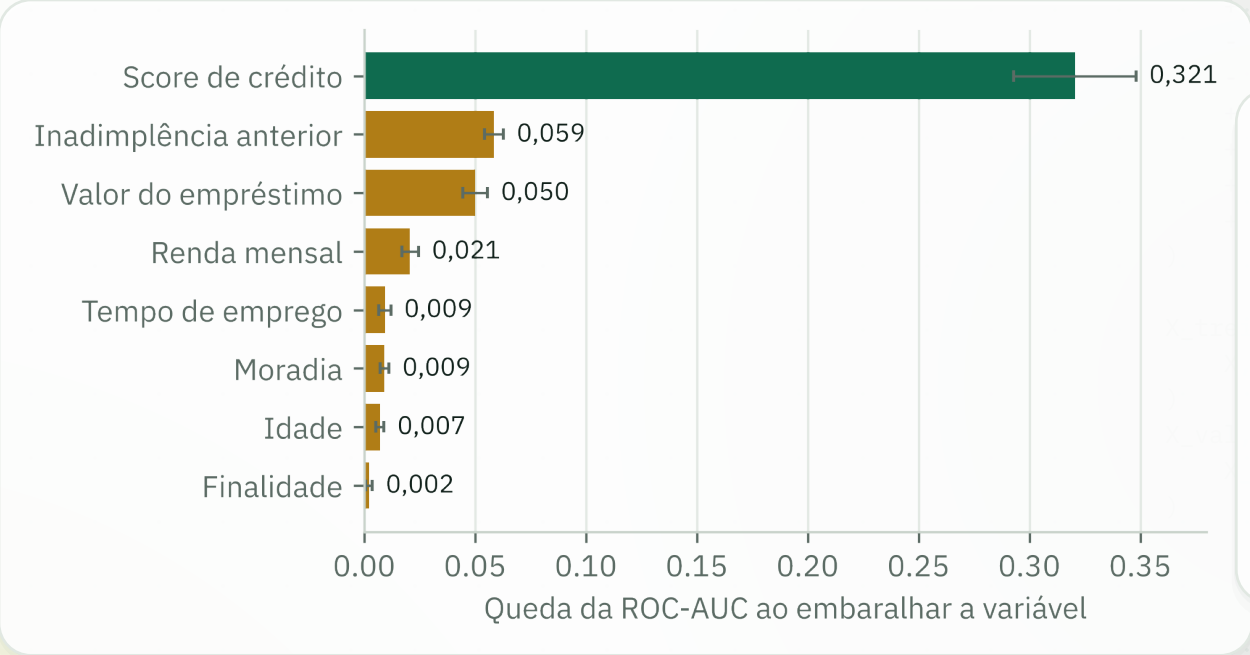
VALIDAÇÃO X TESTE

88,0% x 90,2%: o número é confiável

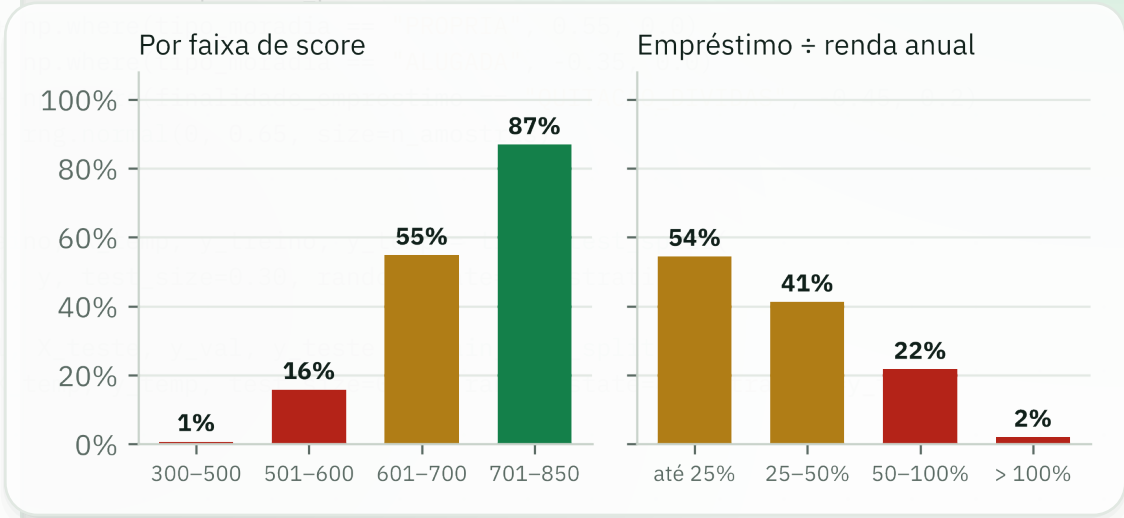
Métricas recalculadas com as versões fixadas no requirements.txt (scikit-learn 1.5.2, numpy 2.1.1, pandas 2.2.3).

LEITURA DO MODELO

O score manda; inadimplência e valor pedido vêm depois



Importância por permutação: quanto a ROC-AUC do teste cai quando embaralhamos cada variável, 15 repetições.

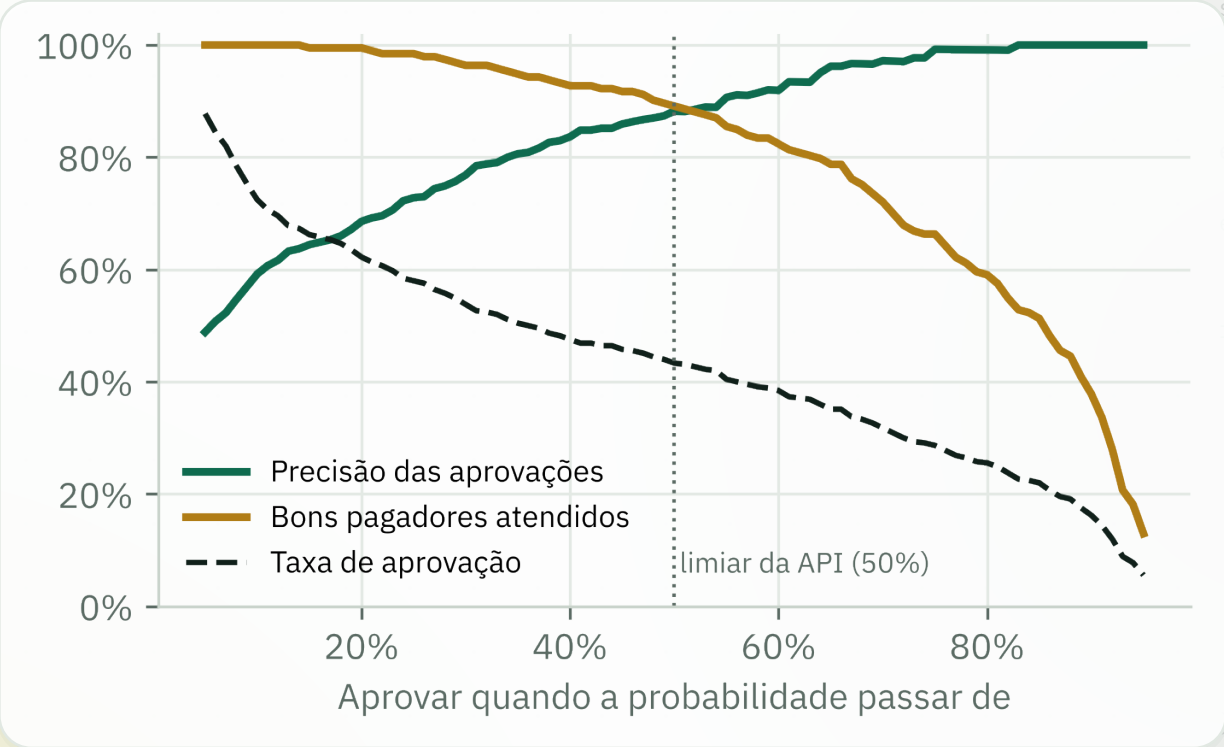


Na base: 87% de aprovação acima de 700 pontos, contra 1% abaixo de 500. Quando o empréstimo passa da renda anual, a aprovação cai para 2%.

O Random Forest aprendeu os mesmos sinais que um analista experiente usaria, com pesos medidos em vez de intuição.

DECISÃO DE NEGÓCIO

O limiar de aprovação é uma política de crédito



A API aprova acima de 50%. Subir o limiar reduz o risco aceito, mas recusa bons clientes.
Números nos 450 clientes de teste.

AGRESSIVA

30%
aprova acima de

Aprovados
54%
Precisão
77%
Bons atendidos
96%
Maus aprovados
56
Bons recusados
7

PADRÃO DA API

50%
aprova acima de

Aprovados
43%
Precisão
88%
Bons atendidos
89%
Maus aprovados
23
Bons recusados
21

CONSERVADORA

70%
aprova acima de

Aprovados
32%
Precisão
97%
Bons atendidos
72%
Maus aprovados
4
Bons recusados
54

Matriz de confusão no limiar de 50%

Bom pagador	**172** aprovados certos	**21** bons recusados
Mau pagador	**23** maus aprovados	**234** recusados certos

Cada camada tem uma função e um status próprio

{ }
Cliente
Streamlit, cURL,
site ou sistema

→

->
Rota
APIRouter
POST /predict

→

Field
Pydantic
DadosClienteInput
tipos e limites

→

if
Serviço
regras de negócio
credito_service

→

RF
Modelo na RAM
Random Forest
predict_proba

→

200
Resposta
PredicaoCredito
Output (JSON)

↓ **422**
Unprocessable Entity
tipo errado, fora do limite, campo faltando ou extra

↓ **400**
Bad Request
cadastro incoerente: emprego > idade - 14, ou empréstimo > 20x a renda

↓ **200**
OK: decisão, probabilidades, faixa de risco e versão

startup · lifespan
joblib.load() 1 única vez



shutdown
libera a memória

cada barra é uma requisição atendida com o modelo já na memória, sem ler o disco (@asynccontextmanager)

FASE 5 · CONTRATO DA API

Três rotas, um contrato tipado de ponta a ponta

GET	/health	Serviço online? Modelo na RAM? Métricas de teste.
POST	/predict	Avalia 1 cliente e devolve a decisão.
POST	/v1/predict	Mesma lógica, rota versionada.

Entrada validada por DadosClienteInput

idade	inteiro, 18 a 100 (obrigatório)
pontuacao_credito	inteiro, 300 a 850 (obrigatório)
renda_mensal	maior que 0, até 1.000.000
valor_emprestimo	500 a 500.000
tempo_emprego_anos	0 a 60
tipo_moradia, finalidade	Enum: só valores da lista
extra="forbid"	campo desconhecido gera 422

Requisição real

```
POST /predict  Content-Type: application/json
{
  "nome_cliente": "Carlos Eduardo Santos",
  "idade": 35,
  "pontuacao_credito": 720,
  "renda_mensal": 7500.0,
  "valor_emprestimo": 18000.0,
  "finalidade_emprestimo": "NEGOCIOS"
}
```

Resposta real

200 OK

```
{
  "nome_cliente": "Carlos Eduardo Santos",
  "idade_informada": 35,
  "pontuacao_informada": 720,
  "decisao": "Aprovado",
  "codigo_decisao": 1,
  "probabilidade_aprovacao": 0.9498,
  "probabilidade_risco_inadimplencia": 0.0502,
  "faixa_risco": "Baixo Risco",
  "indice_comprometimento_anual": 0.2,
  "versao_modelo": "1.0.0"
}
```

FASE 5 · STATUS HTTP NA PRÁTICA

Quatro pedidos, quatro desfechos diferentes

200

Cliente aprovado

Carlos · score 720, renda R$ 7.500

```
decisao: "Aprovado"
probabilidade_aprovacao: 0.9498
faixa_risco: "Baixo Risco"
```

200

Cliente reprovado

Roberto · score 360, inadimplente, pede 2x a renda anual

```
decisao: "Reprovado"
probabilidade_aprovacao: 0.0029
faixa_risco: "Alto Risco"
```

422

Barrado pelo Pydantic

idade 15, score 990

```
idade: Input should be greater than or equal to 18
pontuacao_credito: Input should be less than or equal to 850
```

400

Barrado pela regra

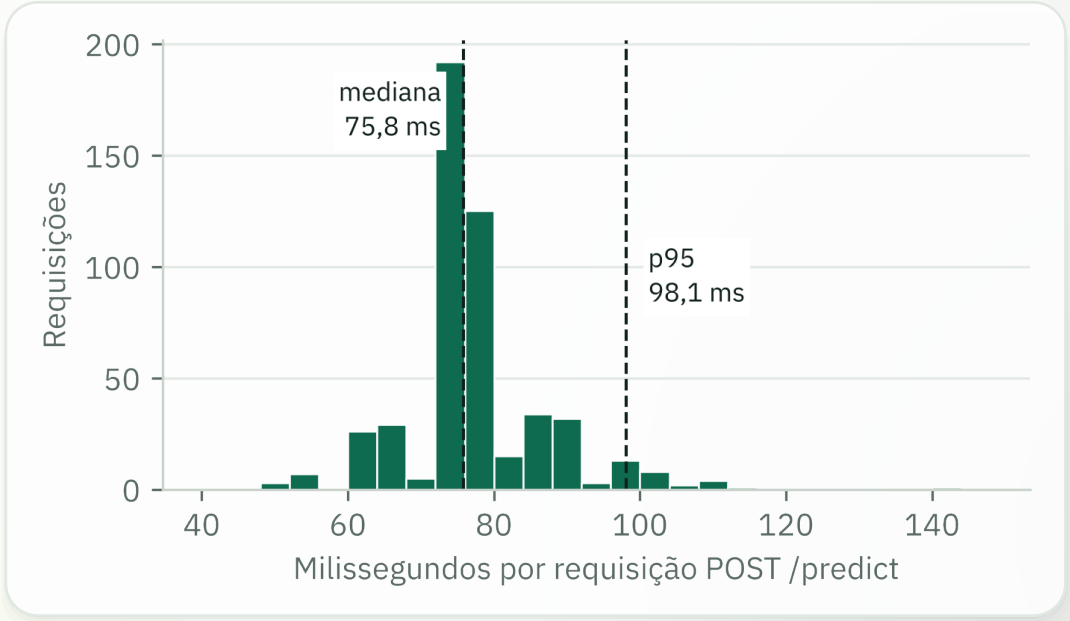
20 anos de idade, 15 anos de emprego

```
Inconsistencia cadastral:
tempo_emprego_anos (15.0) excede
a idade laboral possivel (6 anos).
```

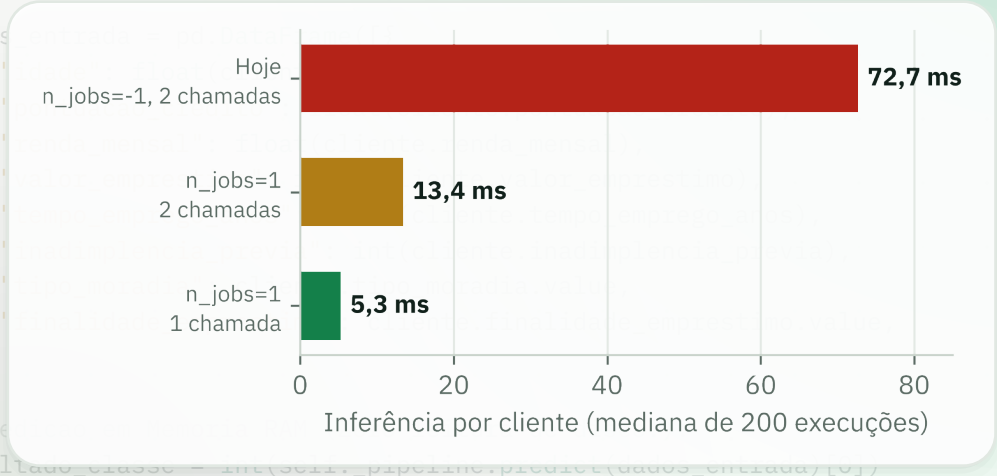
13/13 cenários da bateria com o status esperado na API real (200, 400, 422, 404, 405), e **6/6** testes do pytest passando com as versões do requirements.txt.

PERFORMANCE MEDIDA

A API responde, e pode ficar 13 vezes mais rápida



500 chamadas reais a POST `/predict` com Uvicorn local, 2 CPUs. Mediana 75,8 ms, p95 98,1 ms. O GET `/health` responde em 1,4 ms: o tempo está na predição.



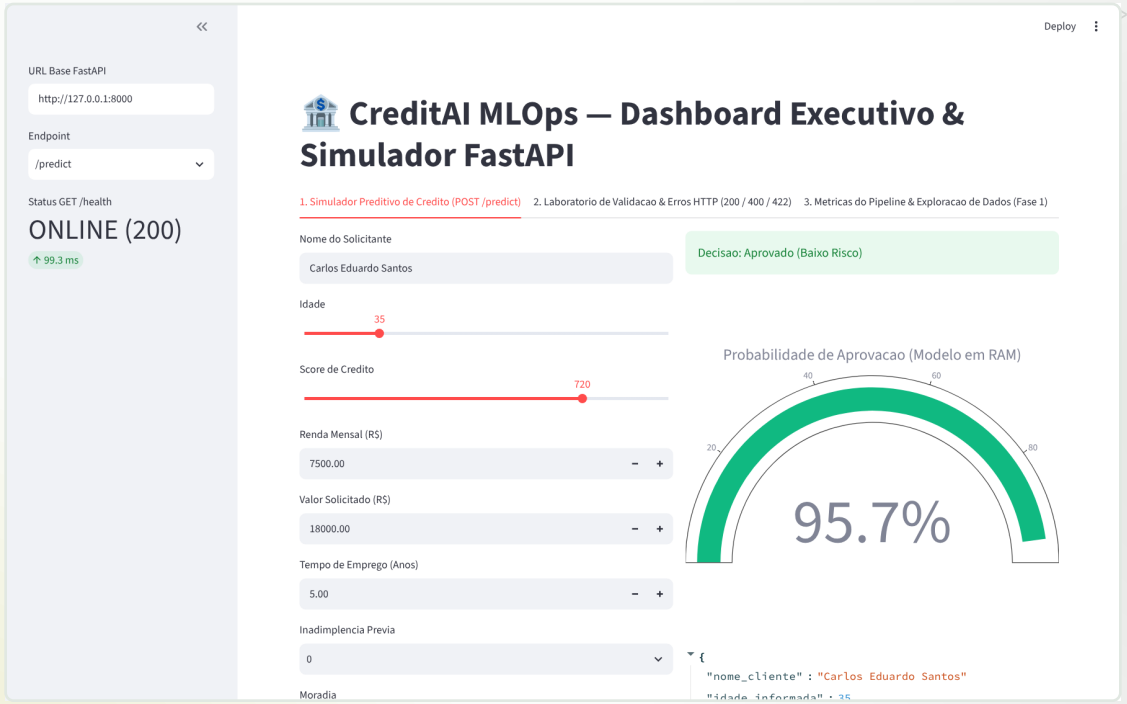

```
# credito_service.py: 1 chamada em vez de 2
- resultado_classe = int(self._pipeline.predict(df)[0])
- probabilidades = self._pipeline.predict_proba(df)[0]
+ probabilidades = self._pipeline.predict_proba(df)[0]
+ resultado_classe = int(probabilidades[1] > probabilidades[0])

# main.py, no lifespan, depois de carregar o modelo
+ pipeline.named_steps["classificador"].set_params(n_jobs=1)
```

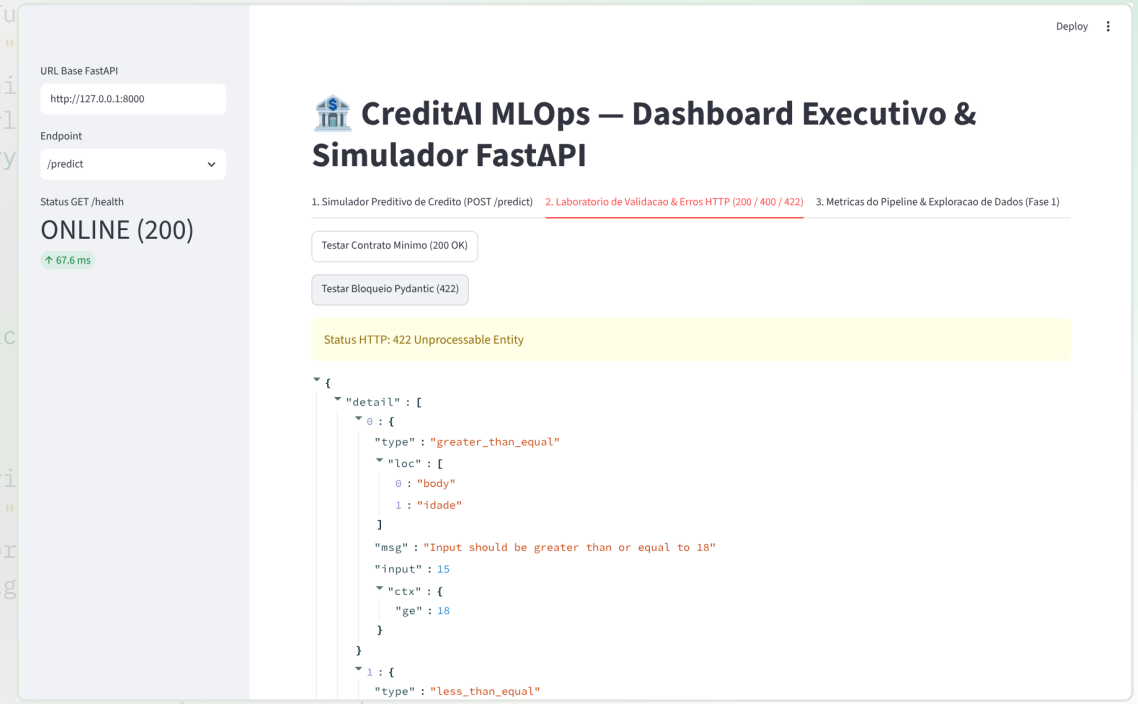
Por que: com `n_jobs=-1`, cada predição de uma única linha abre um conjunto de threads para as 200 árvores. Esse custo é maior do que o cálculo em si.

INTERFACE DO ANALISTA

O painel Streamlit conversa com o modelo só por HTTP



Simulador: GET /health online na barra lateral e POST /predict a cada mudança. O formulário vem com a finalidade Educação: por isso 95,7%, e não os 95,0% do exemplo com Negócios.



Laboratório: o botão de teste envia idade 15 e score 990. A API devolve 422 com cada campo inválido detalhado.

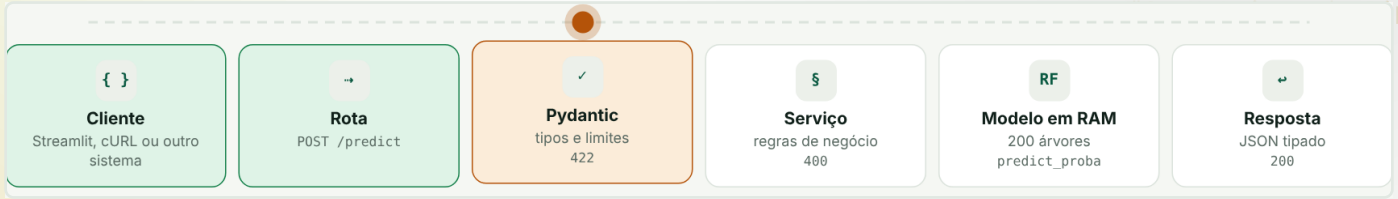
APRESENTAÇÃO INTERATIVA

Dashboard web: o modelo roda no navegador



jorgerodrigues2026.github.io/credito-dashboard

- ✓ As 200 árvores exportadas para JavaScript: **3.000 de 3.000** clientes com a mesma probabilidade do Python.
- ✓ Uma réplica da API devolve o **mesmo JSON** da FastAPI em 26 de 26 requisições testadas.
- ✓ Simulador, fluxo animado da requisição, laboratório REST, política de crédito e modo apresentação.
- ✓ Opção de ligar o simulador na API real, com CORS liberado.



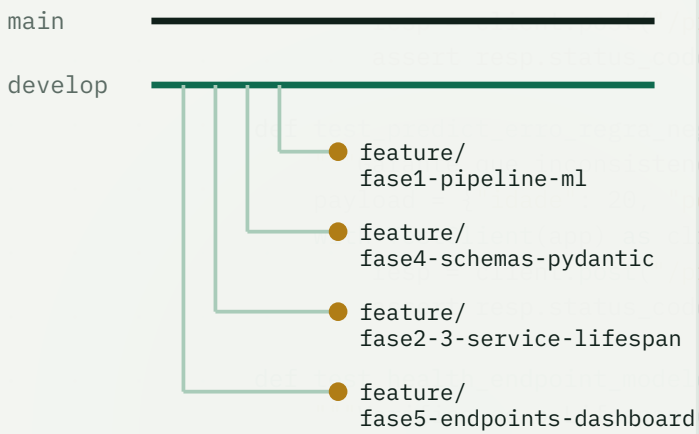
ENGENHARIA E ENTREGA

Camadas, testes e versionamento

Estrutura

```
credito-ml-api/
  app/schemas/cliente.py                422
  app/services/credito_service.py       400
  app/routers/predict_routes.py
  main.py                               lifespan
  train_pipeline.py                     Fase 1
  dashboard.py                          Streamlit
  tests/test_api.py                     6 testes
  data/  models/
  requirements.txt
```

GitFlow



Merge com --no-ff, sem apagar as features.

Testes 6 / 6

- ✓ /health com modelo na RAM
- ✓ /predict aprova Ana
- ✓ /v1/predict reprova Roberto
- ✓ contrato mínimo (idade + score)
- ✓ 422: idade 15, score 950
- ✓ 400: 15 anos de emprego aos 20

```
# ambiente
python -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
# Fase 1: treino e serialização
python train_pipeline.py
# API: http://127.0.0.1:8000/docs        # painel: http://localhost:8501
uvicorn main:app --reload                streamlit run dashboard.py
# testes
pytest -v
```

CONCLUSÃO

O que o projeto entrega e o que vem depois

Entregue

- ✓ Pipeline reprodutível, sem vazamento, com **90,2%** de acurácia e **0,971** de ROC-AUC no teste.
- ✓ API FastAPI em camadas, com o modelo carregado **uma vez** na memória.
- ✓ Contratos Pydantic e regras de negócio com **status HTTP semânticos**.
- ✓ Painel Streamlit e dashboard web para o analista e para apresentação.
- ✓ **6/6** testes e GitFlow com uma branch por fase.

Próximos passos

- ✓ Aplicar a otimização de latência: de ~76 ms para ~5 ms.
- ✓ Reduzir o sobreajuste com `min_samples_leaf` e menos profundidade.
- ✓ Explicar cada decisão ao cliente com SHAP.
- ✓ Monitorar a deriva dos dados e versionar modelos com MLflow.
- ✓ Empacotar em Docker e publicar a API em nuvem.

90,2%
acurácia no teste

0,971
ROC-AUC no teste

~76 ms
por predição hoje, ~5 ms otimizada

26/26
respostas idênticas: site x API real

Do notebook à produção: o mesmo modelo responde ao Streamlit, ao site e a qualquer sistema que fale HTTP, sempre com o mesmo critério e a mesma validação.